

JavaScript Iterators

[Back to JS page](#)

Table of Contents

- [JavaScript Iterators](#)
 - [The Iterator Object](#)
 - [Example 1](#)
 - [The next\(\) Method](#)
 - [Example 2](#)
 - [Iterator.from\(\) Method](#)
 - [Example 3](#)
 - [Iterator Helper Methods](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

The Iterator Object

An iterator is an object that defines a sequence of values and a termination condition. It implements the `next()` method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Iterator Object</h4>
<p id="demo"></p>

<script>
// An iterator is an object with a next() method
// It defines a sequence of values

// Arrays are iterables (have Symbol.iterator)
const arr = [10, 20, 30];
const iterator = arr[Symbol.iterator]();

let text = "Array iterator for [10, 20, 30]:<br><br>";
text += "typeof arr[Symbol.iterator]: " + typeof arr[Symbol.iterator] + "<br>";
text += "Iterator has next(): " + (typeof iterator.next === "function");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The next() Method

The `next()` method returns an object with `value` and `done` properties. When `done` is true, iteration is complete:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The next() Method</h4>
<p id="demo"></p>

<script>
const fruits = ["Apple", "Banana", "Orange"];
const iterator = fruits[Symbol.iterator]();

let text = "Iterating with next():<br><br>";
let result = iterator.next();
while (!result.done) {
  text += "next(): value = " + result.value + ", done = " + result.done + "<br>";
  result = iterator.next();
}
text += "next(): value = " + result.value + ", done = " + result.done + " (done!)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Iterator.from() Method

`Iterator.from()` creates an iterator from an iterable object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterator.from() Method</h4>
<p id="demo"></p>

<script>
let text = "";

if (Iterator.from) {
  // Create an iterator from an array
  const iterator = Iterator.from([1, 2, 3]);

  text += "Iterator.from([1, 2, 3]):<br>";
  let result = iterator.next();
  while (!result.done) {
    text += "value = " + result.value + "<br>";
    result = iterator.next();
  }
} else {
  text += "Iterator.from() is not supported in this browser yet.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Iterator Helper Methods

Newer iterators support helper methods like `map()`, `filter()`, and `take()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Iterators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Iterator Helper Methods</h4>
<p id="demo"></p>

<script>
let text = "";

if (Iterator.from) {
  const iterator = Iterator.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);

  // Using helper methods
  const doubled = iterator.map(v => v * 2);

  text += "Iterator helpers:<br><br>";
  text += "map(v * 2): ";
  let result = doubled.next();
  while (!result.done) {
    text += result.value + " ";
    result = doubled.next();
  }
  text += "<br><br>";
}

text += "<b>Available helper methods:</b><br>";
text += "map(), filter(), take(), drop()<br>";
text += "every(), some(), find(), reduce()<br>";
text += "forEach(), flatMap()";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Iterators](#)